

Retrieval-Augmented Generation (RAG) Architecture Report Author: AI Systems Engineering Group Date: September 2026

1. Executive Summary

Retrieval-Augmented Generation (RAG) is a technique that augments large language model (LLM) prompts with verified external knowledge retrieved from vector databases. By doing so, RAG dramatically reduces hallucinations, guarantees factual accuracy, and allows AI models to answer questions using proprietary or specialized internal documentation.

2. Document Ingestion and Chunking

Document ingestion begins by extracting raw text from document formats such as PDFs, Markdown, and HTML. In our implementation, text is split using a `SemanticStructureChunker`. The chunker preserves natural paragraph and sentence boundaries, avoiding the mid-thought truncations common to fixed-character splitters. A target chunk size of 800 characters with an overlap window of 150 characters is maintained to preserve semantic continuity across chunk boundaries.

3. Vector Embeddings and ChromaDB Storage Each document chunk is transformed into a dense vector embedding using Google Gemini's text-embedding-004 model. Crucially, document chunks are indexed using the task_type RETRIEVAL_DOCUMENT, which trains the embedding projection for optimal storage. Embeddings are saved to ChromaDB, a lightweight, persistent vector database. ChromaDB calculates cosine distance to determine semantic similarity between vectors. Metadata including source file name, page numbers, and chunk indices are attached to each vector record.

4. Two-Stage Retrieval and LLM Reranking When a user submits a query, two retrieval stages execute: First Stage: ChromaDB executes vector search using a RETRIEVAL_QUERY embedding to retrieve the top 8 candidates. Second Stage: A Gemini-powered LLM reranker examines the candidates and scores their direct relevance to the query on a scale of 0 to 10. The top 3 highest-scoring chunks are selected for context synthesis.

5. Conversational Memory and Query Reformulation In interactive chat scenarios, users frequently ask contextual follow-up questions such as "What was the second stage?" or "Can you explain its benefits?". These questions fail in standard vector search because pronouns lack self-contained meaning. To resolve this, our Conversational Memory module maintains multi-turn history and uses Gemini to rewrite contextual questions into standalone search queries before querying ChromaDB.

6. Conclusion and Key Findings The combination of semantic chunking, dual-task Gemini embeddings, ChromaDB vector indexing, LLM reranking, and conversational query rewriting produces a robust, production-grade PDF Q&A system with verified page citations.